

```

    if (tt.p[i] > tt.p[k]) continue;   Ensure we get each edge only once.
    if (tt.p[i] >= npts || tt.p[k] >= npts) continue;   No edges connect-
    sega[n] = tt.p[i];                  ing to fictitious
    segb[n] = tt.p[k];                  points.
    segd[n] = dist(pts[sega[n]],pts[segb[n]]);
    n++;
}
}
Indexx idx(segd);                      Sort the edges by creating an index array.
for (j=0; j<npts; j++) mo[j] = j;      Initialize equivalence relation tree.
n = -1;
for (i=0; i<nspan; i++) {              Add exactly nspan segments.
    for (;;) {                          Loop for the shortest valid segment n.
        jj = j = idx.el(sega,++n);
        kk = k = idx.el(segb,n);
        while (mo[jj] != jj) jj = mo[jj];   Track each end to its highest ances-
        while (mo[kk] != kk) kk = mo[kk];   tor.
        if (jj != kk) {                    The segment is valid only if it connects different
            minsega[i] = j;                  highest ancestors.
            minsegb[i] = k;
            m = mo[jj] = kk;
            jj = j;
            while (mo[jj] != m) {
                tmp = mo[jj];
                mo[jj] = m;
                jj = tmp;
            }
            kk = k;
            while (mo[kk] != m) {
                tmp = mo[kk];
                mo[kk] = m;
                kk = tmp;
            }
            break;                          A segment has been successfully added.
        }
    }
}
}
}

```

CITED REFERENCES AND FURTHER READING:

- de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapters 7 and 11.
- O'Rourke, J. 1998, *Computational Geometry in C*, 2nd ed. (Cambridge, UK: Cambridge University Press), Chapter 5.

21.8 Quadtrees and Octrees: Storing Geometrical Objects

Different from a KD tree is another kind of box tree, usually called a *quadtree* in two dimensions or a *octree* in three dimensions. Yes, we know that it ought to be spelled “octree,” not “octrree,” but the latter usage has become standard. We’ll refer to quadtrees and octrees generically as “QO trees” and thus avoid linguistic controversy.

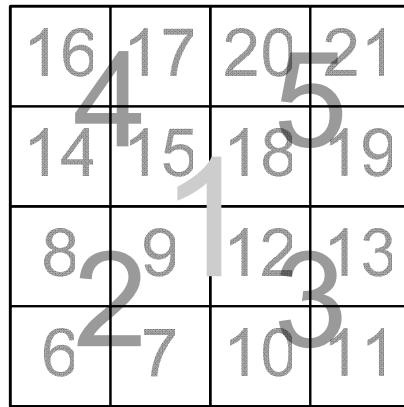


Figure 21.8.1. In a quadtree, the initial square 1 is first subdivided into squares 2, 3, 4, and 5. At the next level of subdivision, 2 is subdivided into 6, 7, 8, 9; 3 into 10, 11, 12, 13; and so forth.

QO trees start with a finite-sized box, usually square or cubical, rather than the near-infinite box used in the KD tree. A QO tree then subdivides each box not in one dimension at a time (as a KD tree) but rather in *all* dimensions. Thus, a square is subdivided into four daughter squares, a cube into eight daughter cubes — quite a brood. The coordinates of the subdivisions are taken to exactly bisect the mother box in each dimension, so all the boxes at one level of the tree are congruent, differing from the original box by a fixed power-of-two factor. Figure 21.8.1 illustrates this in the case of two dimensions.

QO trees thus provide a kind of addressing scheme for two- or three-dimensional space. Accordingly, they can be used to store and retrieve finite-sized geometrical objects that fit into the boxes of the tree at one or another level, and to test for intersections of such objects, for nearness relationships, and so on. The general idea (although there can be variations on this) is to store each object in the smallest box that completely contains it — or, in the case of a zero-sized object like a point, in the appropriate box at the deepest level of tree that we care to implement. Then, when doing a collision or nearness test, we traverse only those parts of the tree that are relevant, much as we did in the applications of KD trees.

Although we will illustrate only the most elementary of applications, QO trees are often at the heart of more complicated algorithms, for example [1-3],

- Hidden polygon removal in the visual plane (which projected polygons intersect a given pixel in the visual field?)
- Fast gravitational or Coulomb N-body calculations (store fictitious objects on various scales that sweep up the multipole moments of the collections of point masses that they contain) [4,5,6]
- Mesh generation (choose a local mesh scale to match the scale in the QO tree at which obstacles or boundaries are stored; the concept of a *balanced* QO tree is often used)
- Image compression (store slowly varying parts of the image as objects high in the tree, and prune unnecessary daughters).

A main weakness of QO trees follows just from their geometrical regularity. If a finite-sized object being stored in a QO tree falls on the boundary between two

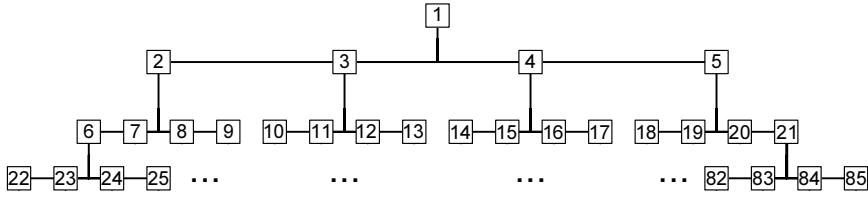


Figure 21.8.2. Quadtree shown in tree form. Because of its regularity, a quadtree’s relationships can be described numerically. For example, the mother of box n is the integer part of $n/4$. The left daughter of box n is $4n - 2$.

boxes of about its size, then it can’t be stored in either of them. Instead, it gets stored in the larger — sometimes very much larger — box that first completely contains it. If N “small” objects are stored, then the number that fall on boundaries of the highest-level boxes scales in two dimensions as $N^{1/2}$, or in three dimensions as $N^{2/3}$. These objects will thus end up stored in just a few boxes at the top of the tree, and they will participate in almost all operations that check for collisions or nearness. Thus, QO trees can usually effect a time savings that turns a naive algorithm scaling as N into one that scales as $N^{1/2}$ (in the two-dimensional case); but only rarely, or with specialized methods, can they get to that Nirvana of $\log N$ or constant scaling. Still, the square- (or, in three dimensions, cube-) root of a large number can be a large factor, and well worth saving in time. So QO trees are good to know about.

That same geometrical regularity of QO trees allows them to be implemented, at least optionally, as an efficient hash structure where most boxes in the tree, if they are empty, require no storage space. We will give such an implementation here, both for its intrinsic advantages and because it is fairly concise to code, using the Hash and Mhash classes from §7.6.

The key observations follow from Figure 21.8.2, which shows a QO tree laid out in tree form. The boxes are numbered as in Figure 21.8.1, starting with box 1 at the root of the tree. With this numbering scheme, there exist simple numerical relations between the mother and daughter boxes. This enables one to navigate the tree — up, down, and sideways — without the use of any stored pointers. In particular, if $k \geq 1$ is a box’s number, the following relationships hold in D (for us, two or three) dimensions:

$$\begin{aligned}
 \text{mother}(k) &= \lfloor (k + 2^D - 2)/2^D \rfloor \\
 \text{leftmost daughter}(k) &= 2^D k - 2^D + 2 \\
 \text{rightmost daughter}(k) &= 2^D k + 1 \\
 \text{total boxes through level } p &= [(2^D)^p - 1]/(2^D - 1)
 \end{aligned}
 \tag{21.8.1}$$

Note that the integer divide by 2^D implied by the $\lfloor \rfloor$ notation can be implemented simply as a right shift by D bits. You should check the formulas in (21.8.1) against Figure 21.8.2 to be sure that you understand how they work. The “levels” of the tree are numbered starting with $p = 1$ for (only) box 1. Notice that the mother of box 1 computes to 0, indicating that it has no mother; this is convenient for testing when to exit loops over ancestry.

Before we get to implementation details for the implementing class `Qotree`, we need to discuss the prerequisites for a class of geometrical objects to be stored

in the tree. Qotree will be templated by a type parameter `elT` representing those objects. To store an object `myel` of type `elT`, you must be sure to provide a method `myel.isinbox()` whose argument is a `Box`, and which returns 1 if `myel` is in the `Box`, or 0 otherwise. Similarly, to erase an object, you need to provide an `==` operator, to decide (by comparison) which is the object to be erased. Those two methods are all that Qotree needs for itself. However, many applications of Qotree (including some that we illustrate later in this section) need either or both of the methods `myel.contains()` and `myel.collides()`, the first returning whether `myel` contains a given point, the second returning whether `myel` collides with another element of type `elT`.

Here is a simple example of a class, representing a circle (when DIM is 2) or sphere (when DIM is 3), that has these methods and can thus be stored and processed with a Qotree:

```
sphcirc.h  template<Int DIM> struct Sphcirc {
Circle (DIM=2) or sphere (DIM=3) object, with methods suitable for use with Qotree.
    Point<DIM> center;
    Doub radius;
    Sphcirc() {}                                Default constructor is needed to make arrays.
    Sphcirc(const Point<DIM> &mycenter, Doub myradius) Construct by explicit center
        : center(mycenter), radius(myradius) {}      and radius.
    bool operator== (const Sphcirc &s) const {        Test if identical.
        return (radius == s.radius && center == s.center);
    }
    Int isinbox(const Box<DIM> &box) {                Is the circle/sphere inside a box?
        for (Int i=0; i<DIM; i++) {
            if ((center.x[i] - radius < box.lo.x[i]) ||
                (center.x[i] + radius > box.hi.x[i])) return 0;
        }
        return 1;
    }
    Int contains(const Point<DIM> &point) {           Is a given point inside the circle/sphere?
        if (dist(point,center) > radius) return 0;
        else return 1;
    }
    Int collides(const Sphcirc<DIM> &circ) {          Does it collide with another circle/sphere?
        if (dist(circ.center,center) > circ.radius+radius) return 0;
        else return 1;
    }
};
```

21.8.1 A Hashed QO Tree Implementation

We will implement the QO tree using two hash memories. First, there is an Mhash multimap memory (called `elhash`) whose keys are box numbers and whose stored elements are the geometrical objects that may be stored in the QO tree, with possibly multiple objects in a single box. Second, there is a single-valued Hash memory (called `pophash`) that associates an integer with every box that either (i) contains one or more elements (is “populated”), or (ii) is an ancestor to a box that is populated. In that integer, bit 0 (least significant bit) is used to indicate whether the box is populated, while bits $1 \dots 2^D$ (that is, $1 \dots 4$ or $1 \dots 8$) are used to indicate which (if any) daughters are themselves either populated or are an ancestor to a populated box. In other words, `pophash`, combined with the relationships in equation (21.8.1), substitutes for the entire structure of doubly linked pointers that might more conventionally implement the tree.

The maximum number of levels $p_{\max}$ that we can represent is limited only by the largest value that can be represented by the integer type that stores box numbers k . Using 32-bit signed integers, 16 levels are possible in two dimensions, since $(4^{16} - 1)/3 < 2^{31} - 1$

(cf. equation 21.8.1). In three dimensions, 11 levels can be represented, since $(8^{11} - 1)/7 < 2^{31} - 1$. Often there is no need for this much resolution ($\sim 10^9$ boxes), so we will provide for setting a smaller value of $p_{\max}$, a good idea since the time to traverse one branch of the tree from root to leaf (a frequently occurring “atom” in other procedures) scales linearly with $p_{\max}$.

```
template<class elT, Int DIM> struct Qotree { qotree.h
    Quadtree (DIM=2) or octree (DIM=3) object to store geometrical objects of type elT.
        static const Int PMAX = 32/DIM;           Roughly how many levels fit in 32 bits.
        static const Int QO = (1 << DIM);         I.e., 4 for quad-, 8 for oct-.
        static const Int QL = (QO - 2);           Offset constant to leftmost daughter.
        Int maxd;
        Doub blo[DIM];
        Doub bscale[DIM];
        Mhash<Int,elT,Hashfn1> elhash;             Contains stored elements hashed by box #.
        Hash<Int,Int,Hashfn1> pophash;             Contains node population info.
        Qotree(Int nh, Int nv, Int maxdep);         The constructor. See below.
        void setouterbox(Point<DIM> lo, Point<DIM> hi); Set scale and position.
        Box<DIM> qobox(Int k);                     Return the box whose number is k.
        Int qowhichbox(elT tobj);                  Return smallest box containing tobj.
        Int qostore(elT tobj);                     Store an elT object in the Qotree.
        Int qoerase(elT tobj);                     Erase an elT object in the Qotree.
        Int qoget(Int k, elT *list, Int nmax);      Retrieve all objects in box k.
        Int qodump(Int *k, elT *list, Int nmax);    Retrieve all objects.
        Int qocontainspt(Point<DIM> pt, elT *list, Int nmax); See below.
        Int qocollides(elT qt, elT *list, Int nmax); See below.
};

template<class elT, Int DIM>
Qotree<elT,DIM>::Qotree(Int nh, Int nv, Int maxdep) :
    elhash(nh, nv), maxd(maxdep), pophash(maxd*nh, maxd*nv) {
    Constructor for a quad- (DIM=2) or oc- (DIM=3) tree that can store a max of nv elements of
    type elT, using hash tables of length nh (typically  $\approx$  nv). maxdep is the number of levels to be
    represented.
    if (maxd > PMAX) throw("maxdep too large in Qotree");
    setouterbox(Point<DIM>(0.0,0.0,0.0),Point<DIM>(1.0,1.0,1.0)); Default scale.
}

template<class elT, Int DIM>
void Qotree<elT,DIM>::setouterbox(Point<DIM> lo, Point<DIM> hi) {
    Sets the scale of Qotree to an outer box defined by points lo and hi. Must be called before
    any elements are stored in the tree.
    for (Int j=0; j<DIM; j++) {
        blo[j] = lo.x[j];
        bscale[j] = hi.x[j] - lo.x[j];
    }
}
```

You will normally call `setouterbox()` immediately after invoking the `qotree` constructor to create a QO tree. Otherwise, you get the default box with a corner at the origin and unit size in each dimension.

Right away, we need two utility routines. The first takes a box’s number (e.g., as in Figure 21.8.1) and returns the actual box (as a `Box<DIM>`). The second takes an object of the type to be stored in the tree (`elT`) and returns the number of the smallest box that contains it. It does this by starting at the top of the tree, trying each possible daughter, and moving deeper into the tree only when a containment is found.

```
template<class elT, Int DIM> qotree.h
Box<DIM> Qotree<elT,DIM>::qobox(Int k) {
    Returns the box indexed by k.
    Int j, kb;
    Point<DIM> plo, phi;
```

```

Doub offset[DIM];
Doub del = 1.0;
for (j=0; j<DIM; j++) offset[j] = 0.0;
while (k > 1) {
    kb = (k + QL) % Q0;
    for (j=0; j<DIM; j++) { if (kb & (1 << j)) offset[j] += del; }
    k = (k + QL) >> DIM;
    del *= 2.0;
}
for (j=0; j<DIM; j++) {
    plo.x[j] = blo[j] + bscale[j]*offset[j]/del;
    phi.x[j] = blo[j] + bscale[j]*(offset[j]+1.0)/del;
}
return Box<DIM>(plo,phi);
}

```

Up through ancestors until get to root.
Which daughter is k? Add its offset.
Replace k by its mother,
where offsets will be twice as big.

At the end, scale the offsets by the final del to
plo.x[j] = blo[j] + bscale[j]*offset[j]/del; make them metrically correct.

Construct the box and return it.

```

template<class elT, Int DIM>
Int Qotree<elT,DIM>::qowhichbox(elT tobj) {
    Return the box number of the smallest box that can contain an element tobj, without regard
    to whether tobj is already stored in the tree.
    Int p,k,kl,kr,ks=1;
    for (p=2; p<=maxd; p++) {
        kl = Q0 * ks - QL;
        kr = kl + Q0 - 1;
        for (k=kl; k<=kr; k++) {
            if (tobj.isinbox(qobox(k))) { ks = k; break; }
        }
        if (k > kr) break;
    }
    return ks;
}

```

Answer is box 1 unless a smaller box found.
Go down through the levels.
Leftmost daughter.
Rightmost daughter.
Do any daughters contain tobj?
No. Therefore, discontinue descent here.

Now we are ready to store elements into the tree, or to erase elements previously stored. With `qowhichbox()`, above, and the methods that belong to the `Mhash`, it is trivial to do the actual store or erase. Trickier to code is to create or erase the trail of “breadcrumbs” in `pophash` that connect the box to its ancestors. When we erase, we must be sure not to cut off the trail to any remaining elements in the same box, or to elements in descendant boxes.

```

qotree.h
template<class elT, Int DIM>
Int Qotree<elT,DIM>::qostore(elT tobj){
    Store the element tobj in the Qotree, and return the box number into which it was stored.
    Int k,ks,kks,km;
    ks = kks = qowhichbox(tobj);
    elhash.store(ks, tobj);
    pophash[ks] |= 1;
    while (ks > 1){
        km = (ks + QL) >> DIM;
        k = ks - (Q0*km - QL);
        ks = km;
        pophash[ks] |= (1 << (k+1));
    }
    return kks;
}

```

Store the element in elhash
and mark its box as populated.
Now leave trail of breadcrumbs to the root mother.
Mother of ks.
Which daughter of km is ks.
Now set the daughter bit in the mother.

```

template<class elT, Int DIM>
Int Qotree<elT,DIM>::qoerase(elT tobj) {
    Erase the element tobj, returning the box number into which it was stored or 0 if the element
    was not found in the Qotree. Note logic very similar to qostore.
    Int k,ks,kks,km;
    Int *ppop;
    ks = kks = qowhichbox(tobj);
    if (elhash.erase(ks, tobj) == 0) return 0;
    if (elhash.count(ks)) return kks;
}

```

Find the box.
It ain't there!
Sisters still in same box, so we are done.

```

ppop = &pophash[ks];
*ppop &= ~((UInt)1);
while (ks > 1) {
    if (*ppop) break;
    pophash.erase(ks);
    km = (ks + QL) >> DIM;
    k = ks - (Q0*km - QL);
    ks = km;
    ppop = &pophash[ks];
    *ppop &= ~((UInt)(1 << (k+1)));
}
return kks;
}

```

Must now erase any unneeded breadcrumbs.
 Unmark the pop bit.
 Up through the ancestors...
 Box is populated or has daughters, so done.
 Erase unneeded (zero) pophash entry.
 Mother of ks.
 Which daughter of km is ks.
 Unset the daughter bit in the mother.

Finally, we need methods to retrieve elements previously stored, either those in a given box (by number), or else all the elements in the tree. In the former case, the Mhash does all the work. In the latter case, however, we must provide the machinery for a recursive search of the tree, since at any stage we may encounter a box with multiple populated daughters. Notice that the calling routine is responsible for supplying storage (as an array list[]) for the result and declaring the maximum number nmax of elements that it is prepared to accept.

```

template<class elT, Int DIM>
Int Qotree<elT,DIM>::qoget(Int k, elT *list, Int nmax) {
    Retrieve all (or up to nmax if it is smaller) elements that are stored in box k of the Qotree. The
    elements are copied into list[0..nlist-1] and the value nlist (≤ nmax) is returned.
    Int ks, pop, nlist;
    ks = k;
    nlist = 0;
    pophash.get(ks,pop);
    if ((pop & 1) && elhash.getinit(ks)) {
        while (nlist < nmax && elhash.getnext(list[nlist])) {nlist++;}
    }
    return nlist;
}

template<class elT, Int DIM>
Int Qotree<elT,DIM>::qodump(Int *klist, elT *list, Int nmax) {
    Retrieve all (or up to nmax if it is smaller) elements that are stored anywhere in the Qotree, along
    with their corresponding box numbers. The elements are copied into list[0..nlist-1] and
    the value nlist (≤ nmax) is returned. The box numbers are copied into klist[0..nlist-1].
    Int nlist, ntask, ks, pop, k;
    Int tasklist[200];
    nlist = 0;
    ntask = 1;
    tasklist[1] = 1;
    while (ntask) {
        ks = tasklist[ntask--];
        if (pophash.get(ks,pop) == 0) continue;
        if ((pop & 1) && elhash.getinit(ks)) {
            while (nlist < nmax && elhash.getnext(list[nlist])) {
                klist[nlist] = ks;
                nlist++;
            }
        }
        if (nlist == nmax) break;
        k = Q0*ks - QL;
        while (pop >= 1) {
            if (pop & 1) tasklist[++ntask] = k;
            k++;
        }
    }
    return nlist;
}

```

gotree.h

Stack of pending box numbers as we recursively traverse the tree.

As long as tasks remain...

Box empty and no daughters.

The box is populated, so we output its contents.

No more room for output!

Leftmost daughter.

Loop over the daughter bits in pop.

Daughter exists. Add to task list.

Next daughter.

The additional functions declared in Qotree pertain to applications, as we now discuss.

21.8.2 QO Tree Elementary Applications

Two important building blocks for applications of QO trees are, first, a routine that returns a list of all stored `elT` elements that intersect (i.e., contain) a specified point; and, second, a routine that returns a similar list of all stored `elT` elements that intersect (i.e., collide with) a specified `elT` element.

An element that intersects a point will evidently be stored in a box that is an ancestor to the box that the point is in, or else in the same box as the point. It takes just one pass down through the levels of the tree to find all such elements.

```
qotree.h  template<class elT, Int DIM>
Int Qotree<elT,DIM>::qocontainspt(Point<DIM>pt, elT *list, Int nmax) {
Retrieve all (or up to nmax if it is smaller) elements in Qotree that contain the point pt. The
elements are copied into list[0..nlist-1] and the value nlist ( $\leq$  nmax) is returned.
    Int j,k,ks,pop,nlist;
    Doub bblo[DIM], bbscale[DIM];
    for (j=0; j<DIM; j++) { bblo[j] = blo[j]; bbscale[j] = bscale[j]; }
    nlist = 0;
    ks = 1;
    while (pophash.get(ks,pop)) {
        if (pop & 1) {
            elhash.getinit(ks);
            while (nlist < nmax && elhash.getnext(list[nlist])) {
                if (list[nlist].contains(pt)) {nlist++;} returning any that contain
                pt.
            }
        }
        if ((pop >>= 1) == 0) break;
        for (k=0, j=0; j<DIM; j++) {
            bbscale[j] *= 0.5;
            if (pt.x[j] > bblo[j] + bbscale[j]) {
                k += (1 << j);
                bblo[j] += bbscale[j];
            }
        }
        if (((pop >> k) & 1) == 0) break;
        ks = QO * ks - QL + k;
    }
    return nlist;
}
```

Start at the top of the tree.
Descend as long as something is there.
The box is populated, so we check its contained elements,
The box has no daughters, so we are done.
Compute k, the single daughter containing pt.
No such daughter exists in the tree.
Daughter exists and is the next node to check.

When an element A intersects another element B , either A and B are in the same box, or else A is in an ancestor box to B , or else B is in an ancestor box to A . Equivalently, for a fixed A , we can find all intersecting B 's by searching A 's box, its ancestors, and its descendants. The latter search requires a task list stack, as we have seen before (e.g., in `qodump`).

```
qotree.h  template<class elT, Int DIM>
Int Qotree<elT,DIM>::qocollides(elT qt, elT *list, Int nmax) {
Retrieve all (or up to nmax if it is smaller) elements in Qotree that collide with an element qt
(which needn't be in the tree itself). The elements are copied into list[0..nlist-1] and the
value nlist ( $\leq$  nmax) is returned.
    Int k,ks,kks,pop,nlist,ntask;
    Int tasklist[200];
    nlist = 0;
    kks = ks = qowhichbox(qt);
    ntask = 0;
    while (ks > 0) {
        Stack of pending box numbers.
        kks saves the starting box.
        Put the starting box and all its ancestors on the
        task list.
    }
```



```

    tasklist[++ntask] = ks;
    ks = (ks + QL) >> DIM;           Move to mother.
}
while (ntask) {
    ks = tasklist[ntask--];
    if (pophash.get(ks,pop) == 0) continue;   Box empty and no daughters.
    if (pop & 1) {                             The box is populated, so we check its contained
        elhash.getinit(ks);                     elements,
        while (nlist < nmax && elhash.getnext(list[nlist])) {
            if (list[nlist].collides(qt)) {nlist++;}   returning any that col-
                                                        lide with qt.
        }
    }
    if (ks >= kks) {                             Recurse only for descendants, not ancestors!
        k = QO*ks - QL;                         Leftmost daughter.
        while (pop >= 1) {
            if (pop & 1)                         Daughter exists. Add to task list.
                tasklist[++ntask] = k;
            k++;                                Next daughter.
        }
    }
}
return nlist;
}

```

As an example of a simple application of a QO tree, let's replicate the functionality of `KDtree::locatenear` (§21.2) with a routine that finds all stored points within a specified radius r of a test point. Using the class `Sphcirc`, points are represented as circles/spheres of zero radius, the test point as a circle/sphere of radius r , and we use `qocollides` to find the collisions.

We implement this application as a structure, `Nearpoints`, whose constructor creates the QO tree out of a vector of points, and whose member function `locatenear` can then be called to find all stored points within any specified radius of any specified point.

```

template <int DIM> struct Nearpoints {                                     qotree.h
    Object for constructing a QO tree containing a set of points, and for repeatedly querying which
    stored points are within a specified radius of a specified new point.
    Int npts;
    Qotree<Sphcirc<DIM>,DIM> thetree;
    Sphcirc<DIM> *sphlist;
    Nearpoints(const vector< Point<DIM> > &pvec)
        : npts(pvec.size()), thetree(npts,npts,32/DIM) {
        Constructor. Creates the QO tree from a vector of points pvec.
        Int j,k;
        sphlist = new Sphcirc<DIM>[npts];
        Point<DIM> lo = pvec[0], hi = pvec[0];   Find bounding box for the points.
        for (j=1; j<npts; j++) for (k=0; k<DIM; k++) {
            if (pvec[j].x[k] < lo.x[k]) lo.x[k] = pvec[j].x[k];
            if (pvec[j].x[k] > hi.x[k]) hi.x[k] = pvec[j].x[k];
        }
        for (k=0; k<DIM; k++) {
            lo.x[k] -= 0.1*(hi.x[k]-lo.x[k]);      Expand it by 10% so that all points
            hi.x[k] += 0.1*(hi.x[k]-lo.x[k]);      are well interior.
        }
        thetree.setouterbox(lo,hi);   Set the tree's outer box and store all the points.
        for (j=0; j<npts; j++) thetree.qostore(Sphcirc<DIM>(pvec[j],0.0));
    }
    ~Nearpoints() { delete [] sphlist; }
    Int locatenear(Point<DIM> pt, Doub r, Point<DIM> *list, Int nmax) {

```

Once the tree is constructed, this function can be called repeatedly with varying points `pt` and radii `r`. It returns `n`, the number of stored points within radius `r` of `pt` (but no larger than `nmax`), and copies those points into `list[0..n-1]`.

```

    int j,n;
    n = thetree.qocollides(Sphcirc<DIM>(pt,r),sphlist,nmax);
    for (j=0; j<n; j++) list[j] = sphlist[j].center;
    return n;
}
};

```

In practice, the routine above is rather slower than `KDtree::locatenear` for this application, because there is a lot of overhead involved in copying `Point` and `Sphcirc` elements around, and in computing `Boxes` as we drill down the tree. By contrast, `KDtree` is lean and mean, since it only stores points and, in our implementation, copies them internally to a fast store of coordinates.

Unlike the KD tree, however, the technique illustrated here can be generalized to much more complicated situations. For example, instead of being simple points, the stored objects could be broadcast reception areas for FM radio stations on a given frequency, and we might want to know where collisions occur with proposed new stations. The `collides()` function between two broadcast areas might involve a lengthy calculation taking into account their powers, the surrounding detailed topography, and so on. In such a case, the overhead of the QO tree might well be negligible as we seek to minimize the number of calls to `collides()`.

As a second example of a simple application, consider a square Petri dish on which spores land, in random positions, one at a time. Each such spore quickly grows into a circular colony that just touches the nearest existing colony (or the edge of the dish), and then stops. (Don't ask us why. This is only an example.) What does the dish look like after N spores have landed?

Rather than give the code in detail, a simple description should suffice: The objects stored in the QO tree are circles. Looping over the number of spores, we pick a random location for each in turn. If the QO tree method `qocontainspt()` indicates that the location lies within an already-stored colony, go on to the next spore. Otherwise, start with a small trial radius and increase it (by doubling, e.g.) until `qotreecollides()` first indicates collisions. Now adjust the trial radius to be the minimum of distances to the colliding elements, add that colony to the tree, and go on to the next spore.

Figure 21.8.3 shows an example of the resulting configuration, after 1000 colonies have grown. (Another 3592 spores landed inside existing colonies and died immediately.)

CITED REFERENCES AND FURTHER READING:

- de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapter 14.[1]
- Samet, H. 1990, *The Design and Analysis of Spatial Data Structures* (Reading, MA: Addison-Wesley).[2]
- Samet, H. 1990, *Applications of Spatial Data Structures: Computer Graphics, Image Processing, and GIS* (Reading, MA: Addison-Wesley).[3]
- Pfalzner, S. and Gibbon, P. 1996, *Many-Body Tree Methods in Physics* (Cambridge, UK: Cambridge University Press).[4]

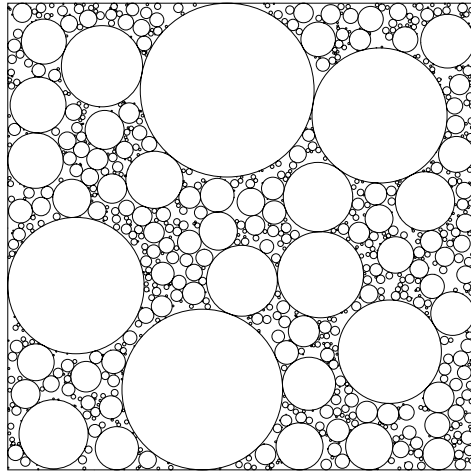


Figure 21.8.3. Spores land randomly on a square (!) Petri dish, and grow to colonies that barely touch the nearest pre-existing colony, or the edge of the dish. A QO tree can be used to keep track of collisions. Here, 1000 colonies have grown to their maximum size.

Greengard, L., and Wandzura, S., eds. 1998, "Fast Multipole Methods," special issue of *IEEE Computational Science and Engineering*, vol. 5, no. 3 (July–September), pp. 16–56.[5]

Gumerov, N.A., and Duraiswami, R. 2004, *Fast Multipole Methods for the Helmholtz Equation in Three Dimensions* (Amsterdam: Elsevier).[6]